

گزارش پروژه درس بینایی کامپیوتر

هدف این پروژه، ترمیم نواحی مخدوش تصاویر (Image Inpainting) با استفاده از شبکه‌های مولد متخاصم (GAN) و معماری بهبود یافته ResUNet است. دیتاست مورد استفاده CIFAR-10 بوده که در آن بخش مربعی کوچکی از مرکز تصویر حذف می‌شود. مدل وظیفه دارد آن بخش را بافت‌دار و سازگار با محیط بازسازی کند. بخش امتیازی انجام شده (معماری آماده استفاده نشده - UI مناسب قرار داده شده). بخش ویدیو خواسته شده در قالب فایل‌های gif در فایل زیپ شده قرار داده شده است.

داده‌ها و پیش‌پردازش

دیتاست مورد بررسی [CIFAR-10](#) بوده که ابعاد آن 32 در 32 پیکسل می‌باشد. پیش‌پردازش داده‌ها شامل نرمال‌سازی مقادیر پیکسل‌ها به بازه $[-1, 1]$ و اعمال ماسک مربعی با ابعاد 16×16 در مرکز تصویر به منظور شبیه‌سازی ناحیه از دست رفته بود. اندازه دسته‌های آموزشی (Batch size) برابر با 128 انتخاب شد. همچنین داده‌ها به دو مجموعه‌ی آموزش و تست تقسیم شدند و برای بارگذاری آن‌ها از کلاس DataLoader با قابلیت جابه‌جایی تصادفی (shuffle) استفاده گردید. کتابخانه‌های اصلی مورد استفاده در این پروژه عبارتند از:

`torch:`

فریم‌ورک اصلی PyTorch برای محاسبه‌های عددی و ساخت شبکه‌های عصبی، پشتیبانی از محاسبات بر روی GPU و مدیریت تنسورها را فراهم می‌کند.

`torch.nn as nn:`

ماژولی برای تعریف لایه‌های پیش‌ساخته شبکه‌های عصبی مانند کانولوشن، نرمال‌سازی، توابع فعال‌سازی و ... کلاس‌ها و بلوک‌های ساختاری مدل از این زیرمجموعه استفاده می‌کنند.

`torch.nn.functional as F:`

شامل توابع سطح پایین‌تر (تابع‌وار) مانند توابع فعال‌سازی، عملیات‌های معیار خطا (مثلاً `relu`, `l1_loss`) و اعمالی که گاهی به صورت مستقیم در محاسبه `loss` یا تبدیل‌ها مورد استفاده قرار می‌گیرند.

`torchvision:`

مجموعه ابزار تکمیلی برای بینایی ماشین که شامل ابزارهای مرتبط با دیتاست‌ها، تبدیل‌های تصویری، و مدل‌های پیش‌آموزش دیده است.

```
torchvision.transforms as transforms:
```

ابزارهایی برای پیش‌پردازش تصویر مثل تبدیل به تانسور، نرمال‌سازی، تغییر اندازه و ... که پیش از ورود به مدل روی تصاویر اعمال می‌شوند.

```
numpy as np:
```

کتابخانه پایه برای محاسبات عددی در پایتون؛ در تبدیل بین فرمت‌ها، دستکاری آرایه‌ها و آماده‌سازی داده‌ها برای برخی عملیات‌های خارج از PyTorch کاربرد دارد.

```
from torch.utils.data import DataLoader, Dataset:
```

ساختارهایی برای بسته‌بندی و بارگذاری داده‌ها به صورت دسته‌ای (batch)، با پشتیبانی از shuffle، چند نخ (multi-worker) و تعریف دیتاست‌های سفارشی از طریق ارث‌بری از Dataset.

```
from skimage.metrics import peak_signal_noise_ratio,  
structural_similarity, mean_squared_error:
```

متریک‌های کمی برای ارزیابی کیفیت بازسازی تصویر

PSNR شدت خطا را نسبت به دامنه‌ی داده می‌سنجد،

SSIM کیفیت ساختاری را مقایسه می‌کند،

MSE متوسط مربع اختلاف پیکسل به پیکسل را می‌دهد.

```
from tqdm import tqdm:
```

ابزار خط پیشرفت (progress bar) برای نمایش بصری پیشرفت حلقه‌های طولانی مانند اپوک‌ها یا دسته‌ها در زمان آموزش استفاده می‌شود.

```
import matplotlib.pyplot as plt:
```

برای رسم نمودارها (مثل loss و معیارها) و نمایش تصاویر نمونه (ورودی، خروجی، ground truth) استفاده می‌شود.

```
import os:
```

برای عملیات روی فایل سیستم مانند ساخت دایرکتوری خروجی (os.makedirs)، ساخت مسیرها و ذخیره سازی چک پوینت ها استفاده میشود.

```
from torchvision.datasets import CIFAR10:
```

دیتاست CIFAR-10 را به صورت آماده فراهم می کند که شامل تصاویر رنگی کوچک در ۱۰ کلاس است و برای آموزش و ارزیابی مدل استفاده می شود.

```
from torchvision.models import vgg19, VGG19_Weights:
```

بارگذاری مدل پیش آموزش دیده VGG19 به منظور استخراج ویژگی های ادراکی (Perceptual Features)؛ VGG19_Weights برای دسترسی به وزن های از پیش آموزش دیده استاندارد استفاده می شود.

```
from torch.nn.utils import spectral_norm:
```

کاربرد برای اعمال Spectral Normalization روی لایه ها (معمولاً روی کانولوشن ها در متمایزگر) به منظور پایداری آموزش GAN با محدود کردن نُرم طیفی وزن ها میباشد.

معماری مدل (از مدل آماده استفاده نشده به علت امتیازی)

مولد (Improved ResUNet Generator)

بخش Encoder شامل چهار بلوک پایین رو (Down Block) است که هر یک از ترکیب لایه های Convolution، Batch Normalization و ReLU تشکیل شده اند. پس از هر لایه کاهشی، یک ResBlock قرار گرفته است تا یادگیری ویژگی های عمیق تر و پایداری بیشتر در انتقال اطلاعات انجام شود. بخش Bridge شامل یک بلوک ResBlock است که نقش اتصال دهنده بین قسمت Encoder و Decoder را ایفا می کند. در بخش Decoder، چهار بلوک بالارو (Up Block) همراه با اتصال های پرشی (Skip Connections) به منظور حفظ جزئیات و بهبود کیفیت بازسازی به کار رفته اند. در نهایت، لایه خروجی یک لایه Convolution با تابع فعال سازی Tanh است که تصویر بازسازی شده را تولید می کند.

متمایزگر (Hinge Patch Discriminator) تصویر بازسازی شده و تصویر ورودی شرطی را به صورت الحاق کانالی دریافت می کند و آن ها را به چندین لایه کانولوشنی مجهز به Spectral Normalization و تابع فعال سازی LeakyReLU وارد می نماید. خروجی این شبکه یک نقشه ی احتمالاتی در سطح Patch است که برای ارزیابی واقع گرایی محلی بخش های مختلف تصویر بازسازی شده مورد استفاده قرار می گیرد.

استخراج ویژگی ادراکی (VGG19 Feature Extractor) با استفاده از شبکه‌ی از پیش آموزش‌دیده VGG19 برای محاسبه‌ی Perceptual Loss انجام می‌شود. در این بخش، تمامی وزن‌های شبکه فریز شده تا در فرآیند آموزش به‌روزرسانی نشوند و تنها از خروجی آن برای استخراج ویژگی استفاده گردد. برای این منظور، ویژگی‌های حاصل از لایه‌های سوم، هشتم، پانزدهم و بیست‌ودوم شبکه انتخاب شده‌اند تا سطوح مختلفی از جزئیات و ساختار تصویر در محاسبه‌ی خطای ادراکی لحاظ شوند.

```
ImprovedResUNetGenerator(  
    (enc1): Sequential(  
      (0): Conv2d(3, 64, kernel_size=(4, 4), stride=(2, 2), padding=(1, 1), bias=False)  
      (1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)  
      (2): ReLU(inplace=True)  
    )  
    (res1): ResBlock(  
      (conv): Sequential(  
        (0): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)  
        (1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)  
        (2): ReLU(inplace=True)  
        (3): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)  
        (4): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)  
      )  
      (activation): ReLU(inplace=True)  
    )  
    (enc2): Sequential(  
      (0): Conv2d(64, 128, kernel_size=(4, 4), stride=(2, 2), padding=(1, 1), bias=False)  
      (1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)  
      (2): ReLU(inplace=True)  
    )  
    (res2): ResBlock(  
      (conv): Sequential(  
        (0): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)  
        (1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)  
        (2): ReLU(inplace=True)  
        (3): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)  
        (4): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)  
      )  
      (activation): ReLU(inplace=True)  
    )  
    (enc3): Sequential(  
      (0): Conv2d(128, 256, kernel_size=(4, 4), stride=(2, 2), padding=(1, 1), bias=False)  
      (1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)  
      (2): ReLU(inplace=True)  
    )  
    (res3): ResBlock(  
      (conv): Sequential(  
        (0): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)  
        (1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)  
        (2): ReLU(inplace=True)  
        (3): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)  
        (4): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)  
      )  
      (activation): ReLU(inplace=True)  
    )  
    (enc4): Sequential(  
      (0): Conv2d(256, 512, kernel_size=(4, 4), stride=(2, 2), padding=(1, 1), bias=False)  
      (1): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)  
      (2): ReLU(inplace=True)  
    )  
    (res4): ResBlock(  
      (conv): Sequential(  
        (0): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)  
        (1): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)  
        (2): ReLU(inplace=True)  
        (3): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)  
        (4): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)  
      )  
      (activation): ReLU(inplace=True)  
    )  
    (bridge): ResBlock(  
      (conv): Sequential(  
        (0): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)  
        (1): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
```

```

        (2): ReLU(inplace=True)
        (3): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
        (4): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    )
    (activation): ReLU(inplace=True)
)
(dec4): Sequential(
  (0): ConvTranspose2d(512, 256, kernel_size=(4, 4), stride=(2, 2), padding=(1, 1), bias=False)
  (1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (2): ReLU(inplace=True)
)
(dec3): Sequential(
  (0): ConvTranspose2d(512, 128, kernel_size=(4, 4), stride=(2, 2), padding=(1, 1), bias=False)
  (1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (2): ReLU(inplace=True)
)
(dec2): Sequential(
  (0): ConvTranspose2d(256, 64, kernel_size=(4, 4), stride=(2, 2), padding=(1, 1), bias=False)
  (1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (2): ReLU(inplace=True)
)
(dec1): Sequential(
  (0): ConvTranspose2d(128, 64, kernel_size=(4, 4), stride=(2, 2), padding=(1, 1), bias=False)
  (1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (2): ReLU(inplace=True)
)
(final): Sequential(
  (0): Conv2d(64, 3, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (1): Tanh()
)
)

```

همانطور که مشاهده می شود، معماری ImprovedResUNetGenerator ترکیبی از ساختار U-Net با بلوک‌های باقیمانده (ResBlock) برای حفظ جزئیات و پایداری آموزش است.

- در ابتدا، چهار بخش کاهشی (enc1 تا enc4) قرار دارند که هر کدام از یک لایه کانولوشن با کرنل 4×4 ، نرمال‌سازی دسته‌ای و فعال‌سازی ReLU تشکیل شده‌اند و به‌صورت متوالی ابعاد فضایی را کاهش می‌دهند و نمایی فشرده‌تر از ورودی می‌سازند.
- بلافاصله پس از هر مرحله کاهشی، یک **ResBlock** قرار دارد (res1 تا res4) که با افزودن مسیرهای باقیمانده و دنباله‌ای از دو کانولوشن 3×3 با نرمال‌سازی دسته‌ای و ReLU، اجازه می‌دهد ویژگی‌های عمیق‌تر بدون افت اطلاعات اصلی استخراج شوند.
- بخش **bridge** نیز همانند یک ResBlock عمل می‌کند و نقطه اتصال بین مسیر کاهشی و مسیر بازبازی است، به‌گونه‌ای که انتقال اطلاعات فشرده‌شده را نرم‌تر کرده و نمای میانی غنی‌تری فراهم می‌آورد.
- در مسیر بازبازی (dec1 تا dec4)، از لایه‌های معکوس کانولوشن (ConvTranspose2d) همراه با نرمال‌سازی دسته‌ای و ReLU استفاده می‌شود تا ابعاد فضایی به تدریج بازسازی شود؛ در هر مرحله از decoder، اتصال‌های پرشی (skip connections) با الحاق خروجی مرحله متناظر در encoder سبب حفظ جزئیات مکانیکی و بافتی می‌گردند.
- در انتها، یک لایه کانولوشن 3×3 به همراه تابع فعال‌سازی Tanh خروجی نهایی تصویر ترمیم‌شده را تولید می‌کند که به دلیل نرمال‌سازی پیشین در بازه‌ی $[-1, 1]$ قرار دارد.

این ترکیب، هم قابلیت تولید بازسازی‌های دقیق از نواحی مخدوش را دارد و هم ثبات در آموزش را از طریق مسیره‌های باقیمانده حفظ می‌کند.

توابع هزینه و معیارهای ارزیابی

توابع هزینه‌ی مورد استفاده در این پروژه شامل Hinge Loss برای آموزش هر دو بخش مولد و متمایزگر، Perceptual Loss مبتنی بر ویژگی‌های استخراج‌شده از شبکه‌ی VGG19، و تابع SSIM Loss به‌منظور بهبود شباهت ساختاری میان تصویر بازسازی‌شده و تصویر اصلی، و L1 Pixel Loss که تنها بر روی ناحیه‌ی حذف‌شده اعمال می‌شود، هستند. این ترکیب از توابع هزینه باعث می‌شود مدل هم از نظر بصری و هم از نظر ساختاری خروجی با کیفیت‌تری تولید کند. در ادامه توضیحات دقیق‌تری راجب این توابع ارائه شده است:

تابع خطای Hinge برای متمایزگر (Discriminator) به گونه‌ای طراحی شده که متمایزگر را تشویق می‌کند تا نمونه‌های واقعی را با مقدار خروجی بزرگ‌تر یا مساوی ۱ و نمونه‌های جعلی را با مقدار خروجی کوچک‌تر یا مساوی -۱ طبقه‌بندی کند. در این تابع از عملیات ReLU استفاده شده تا فقط مواردی که شرط‌ها نقض شده‌اند، جریمه شوند. هدف آن افزایش فاصله بین نمونه‌های واقعی و جعلی در فضای خروجی متمایزگر می‌باشد.

تابع خطای Hinge برای مولد (Loss آدامورسیال) مولد سعی می‌کند متمایزگر را فریب دهد تا نمونه‌های تولیدی را واقعی تشخیص دهد. بنابراین هدف مولد افزایش خروجی متمایزگر روی نمونه‌های تولیدی است. مولد می‌کوشد مقدار آن را بزرگ‌تر کند تا متمایزگر به اشتباه نمونه تولیدی را واقعی تشخیص دهد.

تابع خطای SSIM یا همان (Structural Similarity Index) میزان شباهت ساختاری بین تصویر واقعی و تصویر تولیدی را اندازه‌گیری می‌کند. مقدار SSIM بین -۱ تا ۱ است که مقدار ۱ نشان‌دهنده شباهت کامل است. در اینجا برای تبدیل به تابع خطا از عبارت $SSIM - 1$ استفاده شده که عددی بین ۰ و ۱ خواهد بود. هدف آن حفظ ساختار و کیفیت بصری تصویر تولیدی مشابه تصویر اصلی می‌باشد.

تابع خطای Perceptual تفاوت ویژگی‌های استخراج‌شده از لایه‌های خاص یک شبکه عصبی (مثلاً شبکه‌های پیش‌آموزش‌دیده) را بین تصویر واقعی و تصویر تولیدی اندازه می‌گیرد. به جای مقایسه مستقیم پیکسل‌ها، تفاوت در سطح ویژگی‌های عمیق بررسی می‌شود که بافت و جزئیات مهم‌تر را بهتر نشان می‌دهد. هدف آن افزایش کیفیت و واقع‌گرایی تصویر تولیدی در سطح ویژگی‌های پیچیده‌تر.

تابع خطای L1 Pixel (فقط روی ناحیه حذف‌شده یا Hole) فقط روی ناحیه‌ای از تصویر که حذف یا مخدوش شده (معمولاً ماسک شده با MMM) محاسبه می‌شود تا تمرکز اصلی روی بازسازی آن بخش باشد.

$$L_D^{\text{hinge}} = \frac{1}{2} (\mathbb{E} [\text{ReLU}(1 - D(x, c))] + \mathbb{E} [\text{ReLU}(1 + D(\hat{x}, c))])$$

$$L_{\text{adv}} = -\mathbb{E} [D(\hat{x}, c)]$$

$$L_{\text{SSIM}} = \mathbb{E} \left[\frac{1 - \text{SSIM}(x, \hat{x})}{2} \right]$$

$$L_{\text{perceptual}} = \sum_i \|\phi_i(x) - \phi_i(\hat{x})\|_1$$

$$L_{\text{pixel}} = \|(1 - M) \odot (\hat{x} - x)\|_1$$

در این فرمول‌ها، x تصویر اصلی، $\hat{x}$ تصویر تولیدشده (بازسازی‌شده)، c شرط (مثلاً تصویر ماسک‌شده)، M ماسک، ϕ_i ویژگی‌های استخراج‌شده از لایه‌های مشخص‌شده در VGG19 است.

مجموع وزن‌دار توابع خطای مختلف که مولد باید بهینه کند تا هم کیفیت بصری و هم واقعی بودن نمونه‌های تولیدی حفظ شود. تابع نهایی مولد به شکل زیر ترکیب شد:

$$Loss_G = 1.0 \times L_{\text{adv}} + 0.1 \times L_{\text{perceptual}} + 0.5 \times L_{\text{SSIM}} + 0.9 \times L_{\text{pixel}}$$

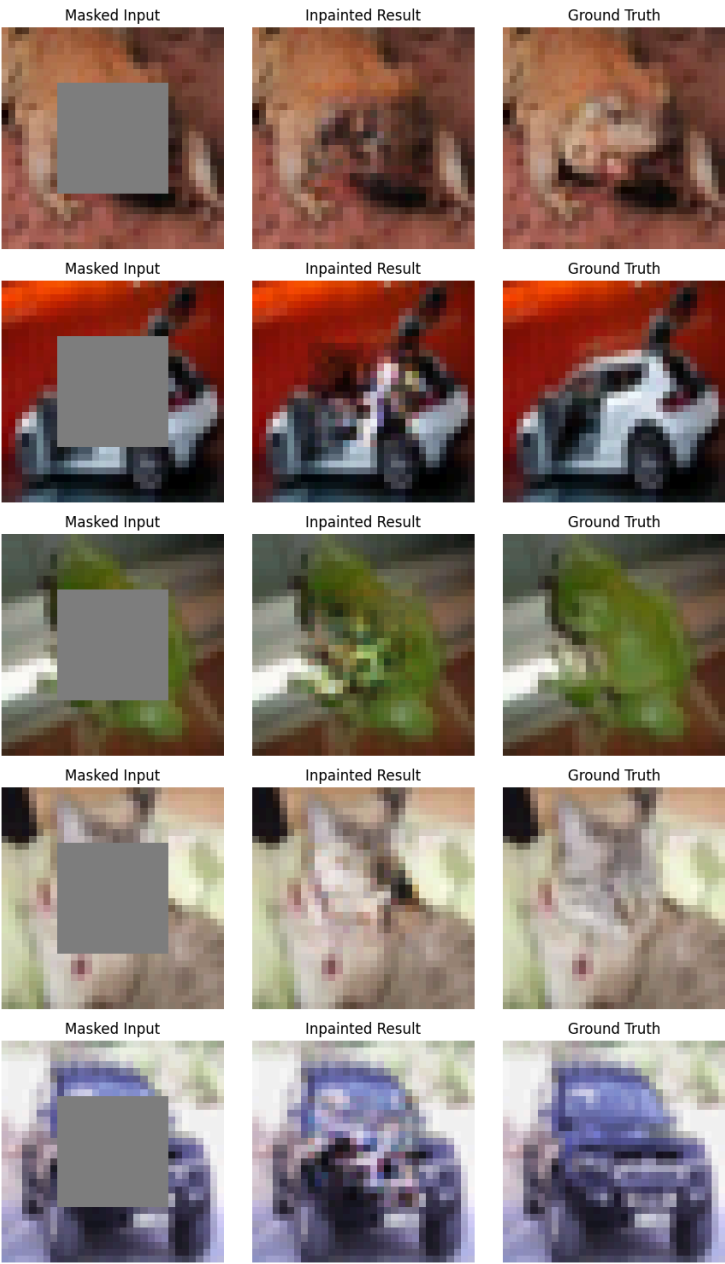
آموزش مدل

فرآیند آموزش مدل در این پروژه طی ۷۵ دوره (epoch) انجام شد. برای بهینه‌سازی، از الگوریتم Adam با مقادیر $\beta = (0.5, 0.999)$ استفاده گردید. نرخ یادگیری برای بخش مولد برابر با 2×10^{-4} و برای متمایزگر 3×10^{-4} در نظر گرفته شد.

به منظور بهبود روند همگرایی و کاهش احتمال گیر افتادن در کمینه‌های محلی، از زمان‌بندی نرخ یادگیری بر اساس روش CosineAnnealingWarmRestarts استفاده شد. این روش باعث نوسان کنترل‌شده نرخ یادگیری در طول آموزش می‌شود و با بازتنظیم‌های دوره‌ای، مدل را در جستجوی بهینه‌ی سراسری یاری می‌کند. وزن‌دهی اولیه لایه‌های Convolution و BatchNorm به صورت Normal Initialization انجام گرفت. در طول آموزش، مدلی که بهترین مقدار PSNR را کسب می‌کرد به عنوان مدل برتر ذخیره می‌شد و علاوه بر آن، در هر ۵ دوره یک Checkpoint کامل از وضعیت آموزش ذخیره گردید.

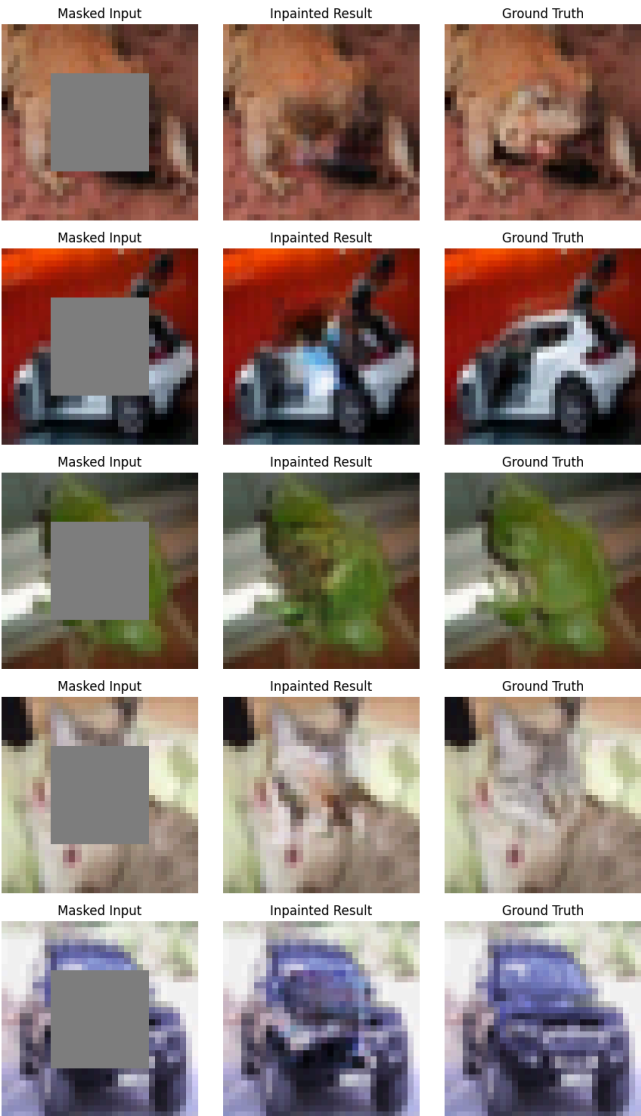
در ادامه نمونه‌های خروجی شامل ورودی ماسک‌شده، تصویر ترمیم‌شده و تصویر اصلی نمایش داده شد.

15th Epoch



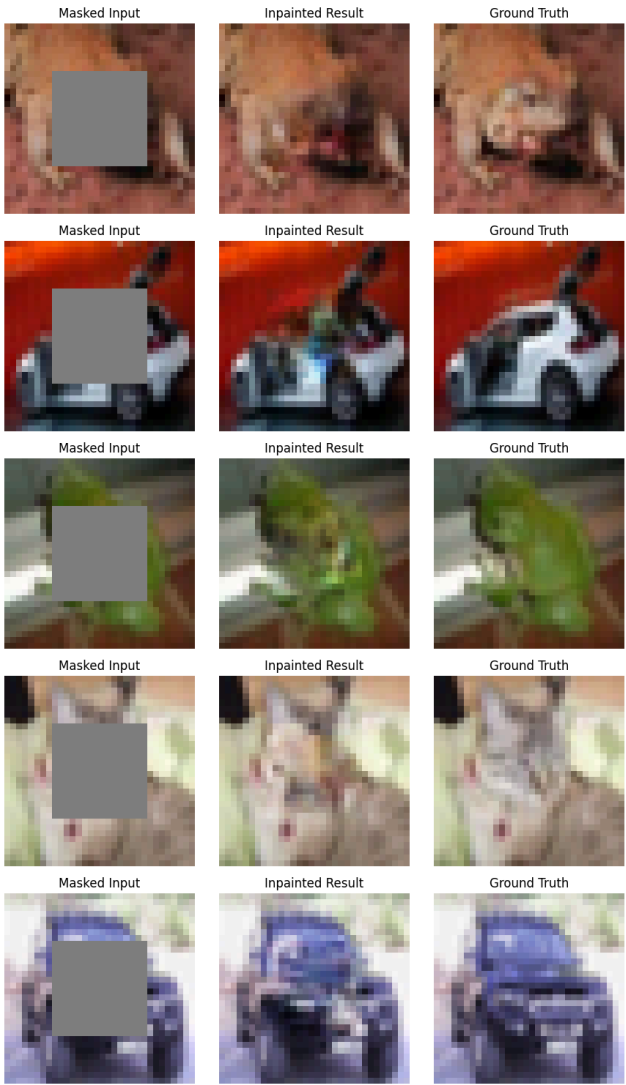
Metric	
PSNR	20.79
SSIM	0.743
MSE	0.00977

35th Epoch



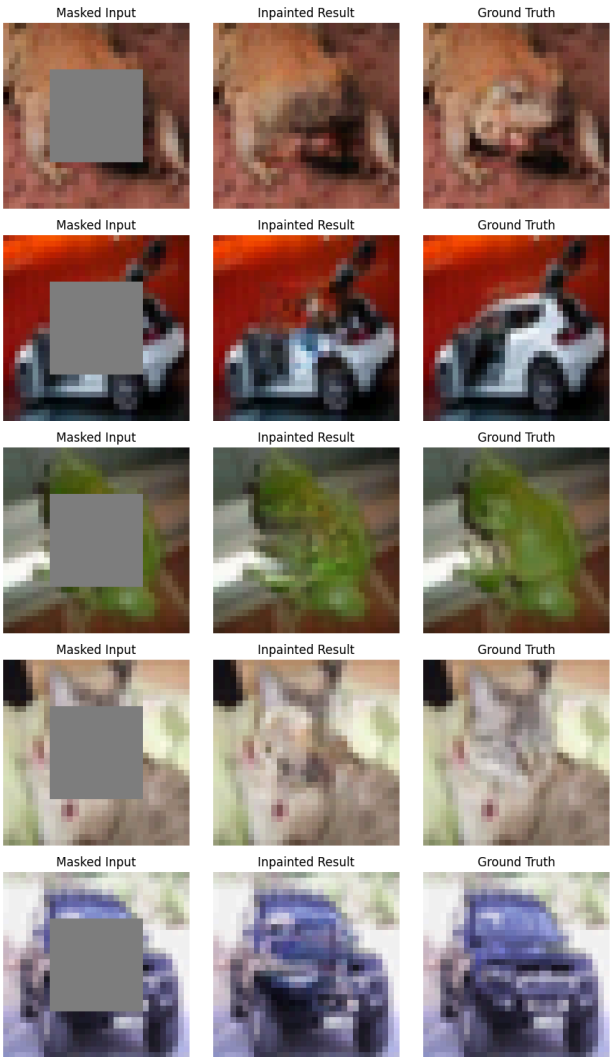
Metric	
PSNR	20.97
SSIM	0.743
MSE	0.00955

55th Epoch

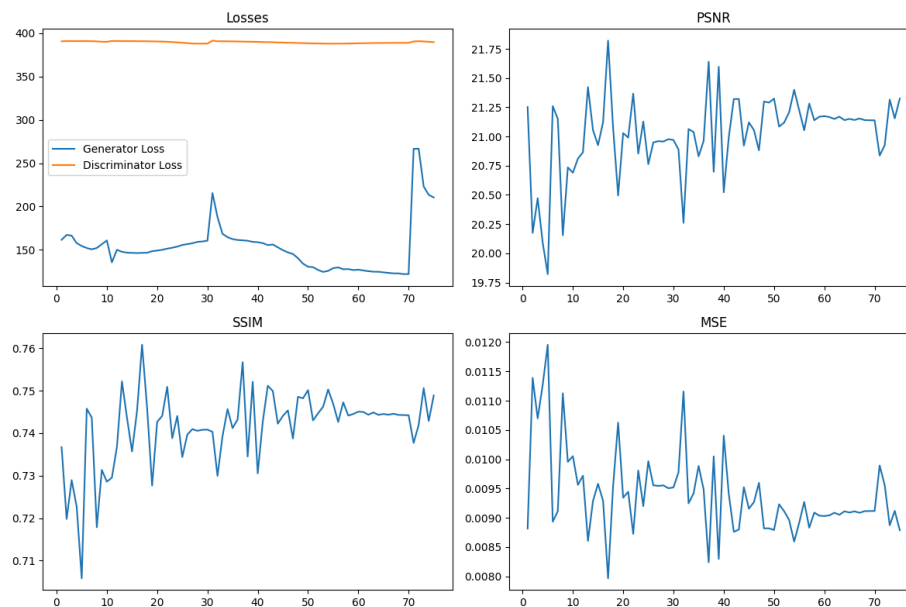


Metric	
PSNR	21.36
SSIM	0.749
MSE	0.00868

75th Epoch



Metric	
PSNR	21.54
SSIM	0.750
MSE	0.00838



بررسی عملکرد روی ویدیو

در این بخش از پروژه، هدف اصلی بازسازی نواحی گم‌شده یا مخدوش‌شده در تصاویر و ویدیوها بود. برای این منظور، از یک مدل ImprovedResUNetGenerator که از قبل آموزش داده شده، استفاده شد که قادر است بخش‌های ناقص تصاویر را به‌طور طبیعی بازسازی کند.

در ابتدا، چندین تصویر از یک دیتاست مورد بررسی قرار گرفت. در این تصاویر، بخش‌هایی از تصویر به‌طور عمدی حذف شده بودند تا مدل بتواند نواحی گم‌شده را تکمیل کند. مدل با دریافت این تصاویر ناقص، اقدام به بازسازی نواحی گم‌شده کرده و تصویر نهایی را تولید کرد. برای نمایش این فرآیند، سه نسخه از هر تصویر تولید شد: نسخه اصلی (بدون تغییر)، نسخه ناقص (پس از اعمال ماسک) و نسخه بازسازی‌شده. این سه تصویر کنار هم قرار گرفتند تا تفاوت‌ها و کیفیت بازسازی به‌طور واضح قابل مشاهده باشد. تمامی این تصاویر در قالب یک GIF ذخیره شدند تا فرآیند بازسازی به‌صورت یکپارچه و واضح نمایش داده شود.

بعد از آن، توجه به ویدیوها جلب شد. ویدیوها به‌عنوان ورودی به مدل ارسال شدند و هر فریم از ویدیو به‌طور جداگانه پردازش شد. مانند تصاویر، فریم‌های ویدیو نیز با اعمال یک ماسک مخدوش شدند و مدل وظیفه بازسازی نواحی گم‌شده را بر عهده گرفت. در نهایت، فریم‌های بازسازی‌شده در یک ویدیوی جدید ترکیب شدند تا بتوان کیفیت بازسازی را در سطح ویدیو نیز ارزیابی کرد. علاوه بر ویدیو، یک GIF از فریم‌های بازسازی‌شده نیز تولید شد تا نمایش بهتری از عملکرد مدل فراهم شود.

این روند، که ابتدا از تصاویر شروع شد و سپس به ویدیوها گسترش یافت، به طور کلی فرآیند بازسازی و ارزیابی مدل را به روشی بصری و قابل درک ارائه داد. نتایج به دست آمده در این مرحله از پروژه امکان مقایسه عملکرد مدل در بازسازی نواحی گم شده را در دو قالب تصویری و ویدیویی فراهم کرد و ابزار خوبی برای ارزیابی دقیق تری از مدل در اختیار قرار داد.

توابع این بخش (اینها صرفاً جهت توضیحات جزئیات کد است و کلیت کارکرد در بخش بالاتر توضیح داده شده):

تابع `make_sample_gif`: تابع `make_sample_gif` برای ایجاد یک GIF از نمونه هایی از دیتاست طراحی شده است که عملکرد مدل را در بازسازی نواحی گم شده نشان می دهد. در این تابع، چند تصویر از دیتاست تست انتخاب می شود و برای هر تصویر، مدل اقدام به بازسازی ناحیه مخدوش شده می کند. خروجی شامل سه تصویر است: تصویر اصلی، تصویر مخدوش شده و تصویر بازسازی شده. این سه تصویر کنار هم قرار می گیرند و در قالب یک GIF ذخیره می شوند. این GIF به عنوان یک ابزار بصری برای ارزیابی نحوه عملکرد مدل در بازسازی تصاویر گم شده عمل می کند و امکان مشاهده سریع و راحت تفاوت ها بین تصاویر اصلی و بازسازی شده را فراهم می آورد.

تابع `inpaint_video_and_gif`: تابع `inpaint_video_and_gif` فرآیند مشابهی را برای ویدیوها انجام می دهد. در اینجا، ابتدا یک ویدیو بارگذاری می شود و سپس هر فریم آن به صورت جداگانه پردازش می شود. برای هر فریم، مدل نواحی گم شده را بازسازی کرده و سه نسخه از فریم تولید می شود: نسخه اصلی، نسخه ناقص و نسخه بازسازی شده. این فریم ها کنار هم قرار می گیرند و در یک ویدیو ذخیره می شوند تا عملکرد مدل در سطح ویدیو نیز به نمایش درآید. علاوه بر ویدیو، یک GIF نیز از فریم های بازسازی شده تولید می شود که می تواند به عنوان پیش نمایشی از ویدیو در دسترس باشد. این تابع به ویژه برای ارزیابی بازسازی در ویدیوها مناسب است و به طور واضح فرآیند بازسازی را در یک قالب پویا و قابل مشاهده به نمایش می گذارد.

نمونه خروجی استفاده از الی (بخش امتیازی)

Upload (1)

